ENTERPRISE ARCHITECTURE AND BUSINESS LOGIC

Domain-Driven Design

Domain-Driven Design (DDD): A software engineering approach that aims to model how businesses work.

DDD ultimately represents the business environment as a collection of **conceptual** classes that each represent some real-world entity.

Domain experts: Subject matter experts in the business domain who provide the business expertise that engineers ultimately transform into software requirements.

Business domain: The value-generating service a company offers to its customers.

Subdomains: Discrete business areas within a business domain that work together to fulfill a business's goals.

Core subdomains: Represent a company's value to customers, and can be considered its reason for being.

Generic subdomains: Facilitate business without being a source of competitive advantage.

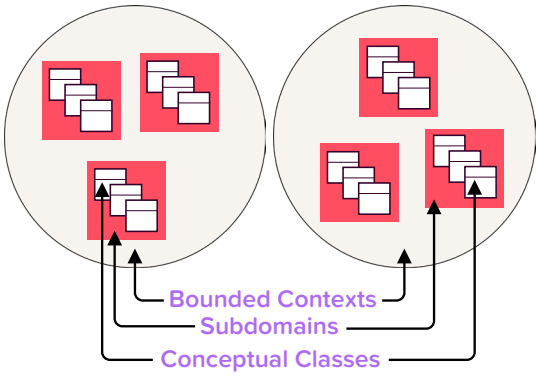
Supporting subdomains: Support a core subdomain.

Ubiquitous language: A common vocabulary agreed upon by stakeholders, which leads to a shared understanding across teams, reducing confusion and slowdowns.

Even with a ubiquitous language, some concepts may have different meanings depending on where they're used. For that reason, designers create **bounded contexts** in their models to set boundaries within which the meaning of a term is agreed upon.

Business-domain model: A visual map illustrating key concepts within the domain.

The business-domain model reflects all relevant business units and shows cause-and-effect relationships.



Important: Software engineers place their greatest emphasis on developing core subdomains into successful systems.

Remember: The activities of generic subdomains are done largely the same way across many industries. Because of that, their development is often outsourced.

A Closer Look: Like generic subdomains, supporting subdomains don't provide any significant competitive edge. Unlike generic subdomains, they require specialization and differentiation.

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Software Architecture

Software architecture: The organization of components in a software system.

Modules: Divide the total functionality of the system into relatively independent parts.

Patterns offer ways to ensure **separation of concerns** between modules, minimizing linkages and allowing for more freedom in changing any particular module.

Antipatterns: Solutions in principle that cause more harm than good in practice.

Orthogonality: The extent to which each module's responsibilities and functions are exclusive to that module.

Types of Patterns

- Architecture: How software is structured at the most general level, which informs how modules are organized and connected.
- Business logic: How real-world business rules are implemented via data creation, processing, and storage.
- Data source: How persistent data—e.g., in a database—is represented in business logic.
- Presentation: How the user interface is generated and connected to the application.

Remember: A system's architecture should be geared to meet its particular needs, and no single architecture is ideal for every system.

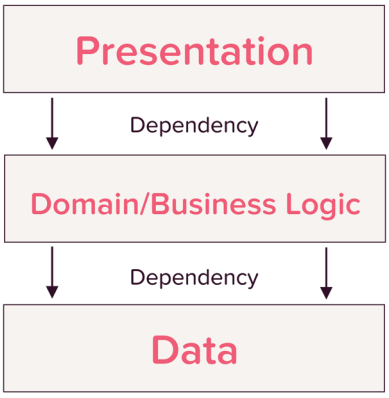
A Closer Look: There are two basic approaches to software architecture design.

- Top down: Scoping the whole, then gradually deconstructing it into more fundamental components.
- Bottom up: Specifying the most fundamental components, then building up and out.

Caveat: When misapplied, even the most useful patterns can become antipatterns.

Layered Architecture Patterns

Layers: An architecture pattern in which a system's modules are arranged as discrete layers in a vertical stack.



©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Presentation layer: Concerns how users interact with the system, which usually involves a graphical user interface (GUI) like a web browser or a smartphone display.

Business-logic layer: Where data is processed according to the business rules discovered during DDD.

Data layer: Manages the storage and retrieval of information processed by the application.

Tiers: The physical locations the software runs on.

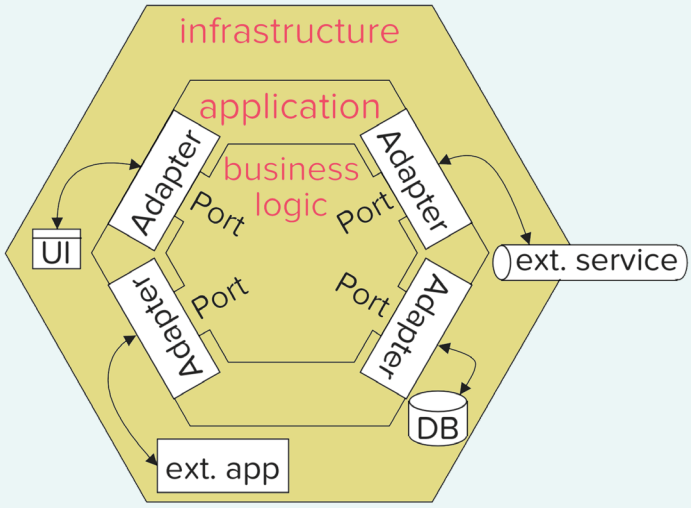
Service layer: Lies between the presentation and business-logic layers, effectively decoupling them.

The service layer orchestrates calls to the business logic so the presentation layer doesn't have to know about business logic, allowing business logic to change without changing presentation

Ports & Adapters: An architecture that makes everything else in the system dependent on the business-logic layer.

Caveat: Sometimes people conflate layers and tiers, but they're different concepts. Layers are simply how the software is organized into modules.

Important: Adapters allow business logic and infrastructure to vary independently.



In the Ports & Adapters pattern, ports, which are defined in the business logic, manage incoming and outgoing data. Adapters, created in the application layer, connect elements like external services to the ports.

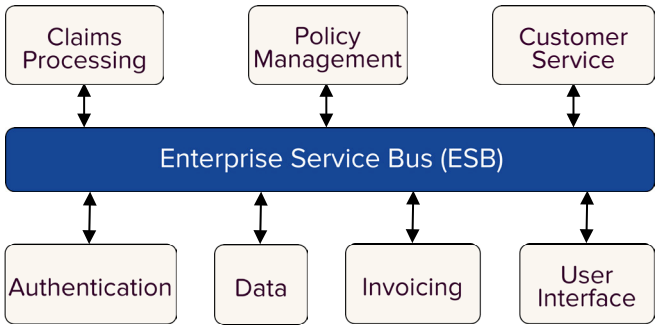
©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Service-Based Architecture Patterns

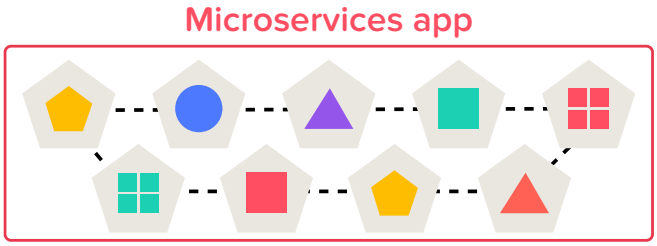
Service-Oriented Architecture (SOA): Decomposes a system into several smaller services that work together. This addresses issues typical of monoliths, including limited scalability, high coupling, and cumbersome integration and deployment.

Services: Modules within SOA that contain all the code and data needed to perform a distinct business function.

Enterprise service bus (ESB): A standard feature of SOA that's responsible for service integration.



Microservices: An approach that breaks down an app's functionality into several services that can be developed and scaled independently.



Microservices vs. SOA

	Microservices	SOA
Share resources	Not generally	Yes
Reuse services	No	Yes

A Closer Look: An SOA service consists of three parts.

1. The contract, or the service's obligations to its consumers and vice versa
2. The interface, or the means by which the service's consumers make requests
3. The implementation, or the service's code

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Business-Logic Patterns

Transactions: Discrete interactions or exchanges involving multiple steps. Transactions are atomic; if one part fails, the system must roll back to the pre-transaction state.

Transaction Script: A pattern that makes each transaction into a separate script or procedure.

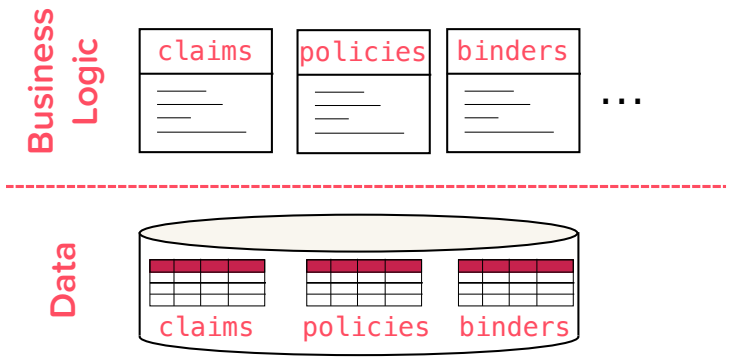
> **Procedure:** A series of statements or commands that orchestrates a transaction, rolls back in the event of failure, and guarantees atomicity.

Domain Model: A pattern that uses conceptual classes as the templates for creating domain objects.

> **Domain object:** Represents an instance of some business entity.
>
> > Domain objects can exist within aggregates, or assemblies of domain objects treated as a whole.
> >
> > **Aggregate root:** The main entry point for accessing and modifying data in the aggregate.

Table Module: A pattern that creates a 1:1 association between business logic and the tables in the database.

> Each table is represented by one class in the business-logic layer that can modify the data in that table, and each of those classes has just one instance.



Tip: Business logic is tied to the domain object, so it doesn't need to be duplicated when the domain object is used in different transactions.

Important: Other parts of the application can interact with the aggregate only through the aggregate root. This ensures data consistency, prevents unintended changes, and guarantees the atomicity of transactions.

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY